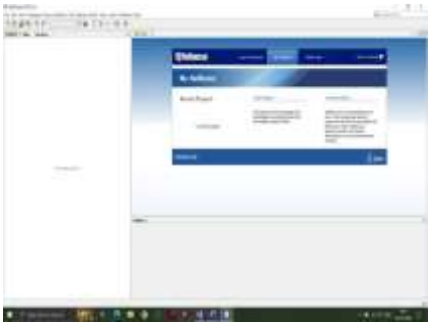
Practical No. 1

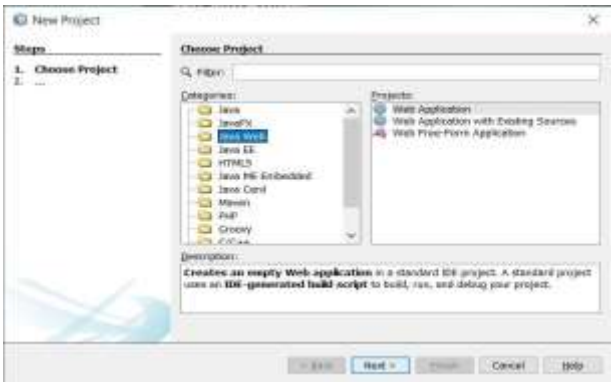
Aim: Create a SOAP Service.

<u>Steps:</u>

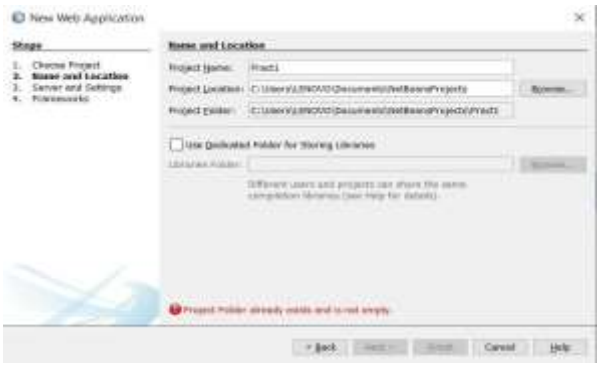
1. Open the NetBeans , and you will get the following screen. Close the start page.



2. Now click on the file tab and click on new project click on new project you will get the following screen :



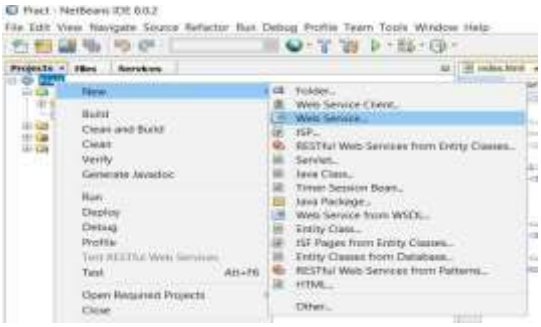
3. In Categories select Java Web and in Projects , Select Web Application, After selecting click on next . You will get the following window:



4. Now Give name to the Project Name: , and click on next . you will get the following window

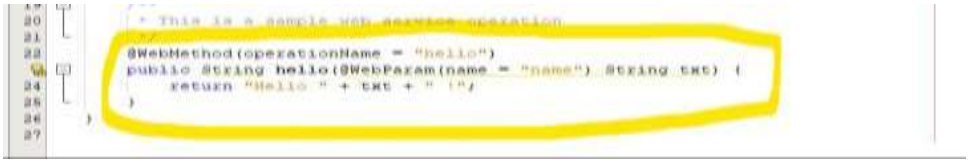
5. thenYou will get the following screen now carefully see in projects section your recently created project appears double click to expand it:

6. Now Right click on the After clicking Web Service following window should appear, now give name to web service



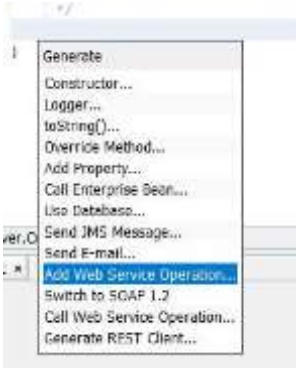
7. project and select new and then select Web Service gain click on finish

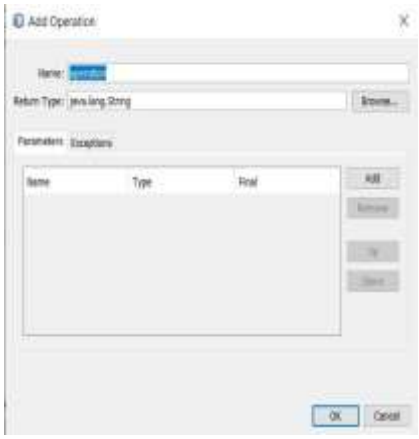
8. After clicking finish you should get the following window erase the mentioned code :



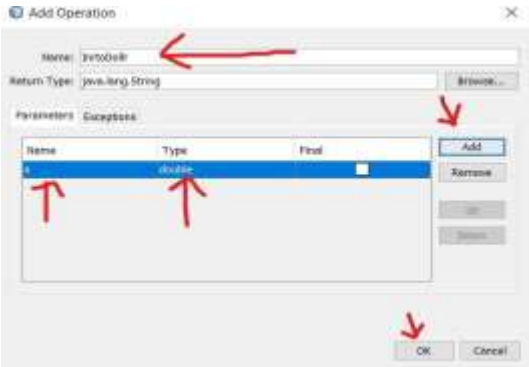
9. The following window would appear :

Just give name to the method or operation and click on add button to add parameters to the method as here we are converting dollar to rupees we should need only one parameter .





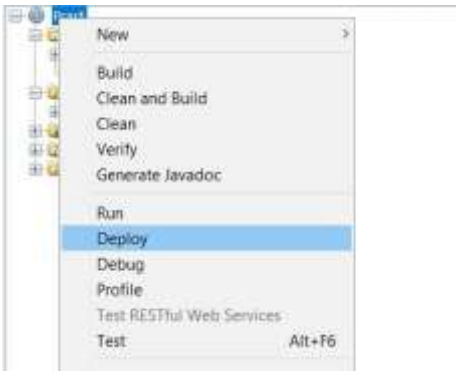
10. Now right click any where and click on insert code and select Add Web Service Operation :



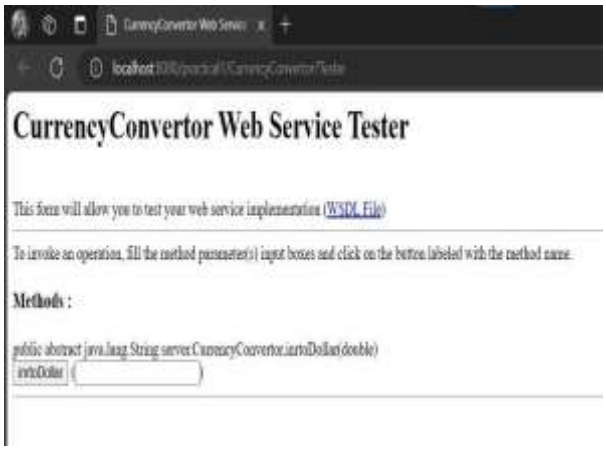
11. After clicking ok code will autogenerate , make changes In that code as mentioned below:

```
@WebMethod(operationName = "InrtoDollar")
public String InrtoDollar(@WebParam(name = "a") double a) {
    //TODO write your implementation code here:
    return "The Indian rupees "+a+" in Dollars is "+(a/83.17);
}
```

12. After clicking ok code will autogenerate , make changes In that code as mentioned below:



13. Now right click on webservice and click on test web service you will get the following



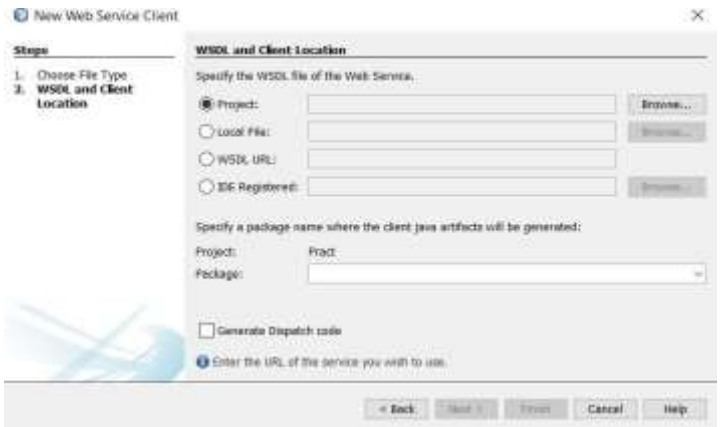
14. Now our web service is successfully deployed. Right click on web pages and select new and select jsp as shown below:

15. Give name and click on finish as shown below do it 2 times on for input and one for

16. In input.jsp create a form for taking user input as shown below:

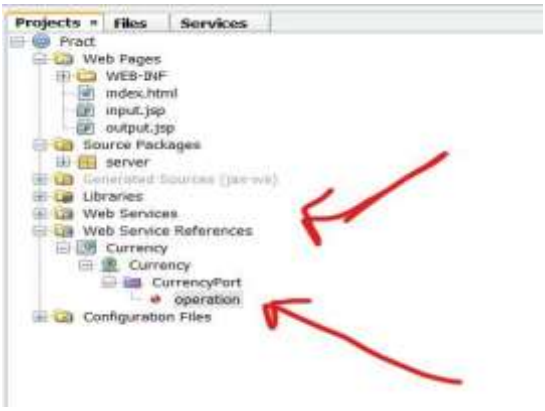
```
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>
        <form action="output.jsp">
            <pre>
                Enter the currency in rupees : <input type="text" name="i1">
                <input type="submit">        <input type="reset">
            </pre>
        </form>
    </body>
</html>
```

17. Now we have to create web service client, for same right click on project and select new then select web service client you will get the following screen: At this step click on browse and select your project and click ok after clicking ok you will get wsdl url as shown below:



18. Now in project section you can se a new folder is been created named “web servicerefrence”.

double click to expand it you should get the following :



19. Now make changes as shown below in the code:

```
<%-- start web service invocation --%><hr/>
<%
try {
    Client.Currency_Service service = new Client.Currency_Service();
    Client.Currency port = service.getCurrencyPort();
    // TODO initialize WS operation arguments here
    double a = Double.parseDouble(request.getParameter("t1"));
    // TODO process result here
    java.lang.String result = port.operation(a);
    out.println(result);
} catch (Exception ex) {
    // TODO handle custom exceptions here
}
%>
<%-- end web service invocation --%><hr/>
```

```
<body>
    <%-- start web service invocation --%><hr/>
<%
try {
    Client.Currency_Service service = new Client.Currency_Service();
    Client.Currency port = service.getCurrencyPort();
    // TODO initialize WS operation arguments here
    double a = 0.0d;
    // TODO process result here
    java.lang.String result = port.operation(a);
    out.println("Result = "+result);
} catch (Exception ex) {
    // TODO handle custom exceptions here
}
%>
<%-- end web service invocation --%><hr/>

</body>
```

20. Now deploy our project again and right click on input.jsp and click run file you will redirect
to a browser page as shown:

21. Enter any numerical value in text box and click on submit you will get the following

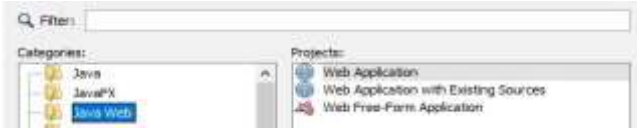
Practical No. 2

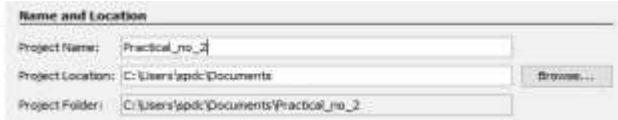
Aim: To create a Simple Web Service that converts the temperature from Celsius to Fahrenheit and vice versa

<u>Steps:</u>

STEP: 1 Create a Web Application:

File → New Project → Java Web → Web Application → Name: Prac2 → Add GlassFish Server 4.1 → Finish → (you'll get index.html, delete this file.

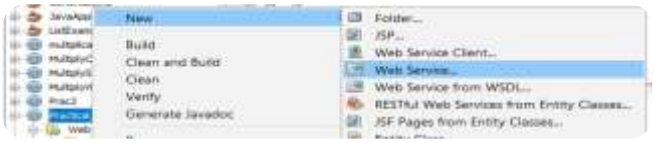




Step 2: Creating a Web_service

Right click on project name → New → Web Service →

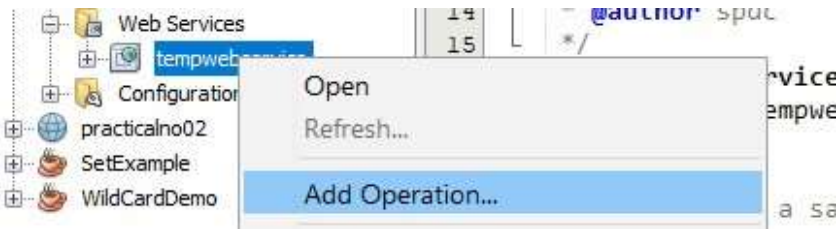
Web servicename = "tempwebservice" → package ="ty" → Finish



Step 3: Add Operations:

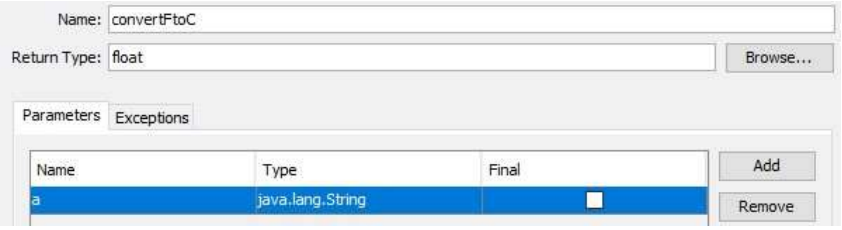
Right click on Web service → tempwebservice → Add Operations → Name:

convertFtoC → Return Type: float → Add a Parameter 'a' with Type: float > OK >



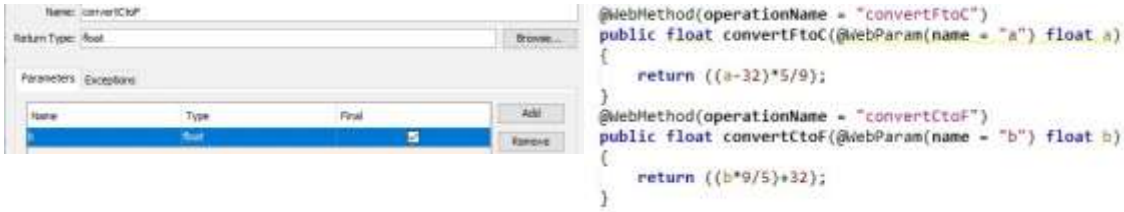
Operation "a"

> Now add → formula in return statement((a-32)*5/9) .



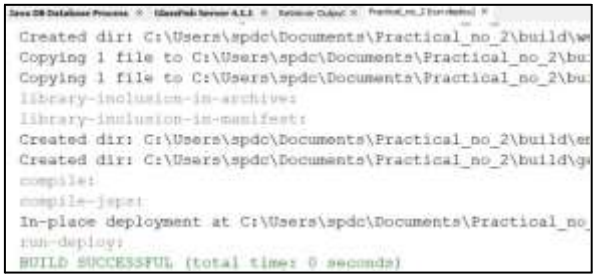
Operation "b"

>Now add →formula in return ((b*9/5)+32).



Deploy:

Make sure the file is deployed → Right click to project name → Deploy



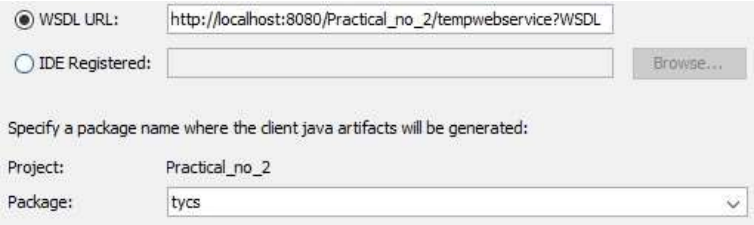
Test web service:

Right click on tempwebservice → test web service → click on WSDL File → Copy the URL

New Web service

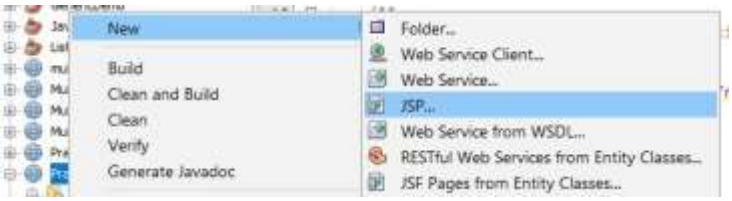
Right click on Web service → Find → others → Web service and Web service client → next

→ paste the copied WSDL File URL→ package = tycs → finish.



JSP Files

Right click on project name → New → JSP file →



Add code in body section of 2nd JSP

```
<body>
    <h1>Hello World!</h1>
    <%
        tycs.Tempwebservice_Service obj = new tycs.Tempwebservice_Service();
        tycs.Tempwebservice port = obj.getTempwebservicePort();

        String val = request.getParameter("txt"); // took the input

        float temp = 0;
        float result1 = 0;
        float result2 = 0;
        if (val != null && !val.isEmpty()) {
            temp = Float.parseFloat(val);        // parse input to float
            result1 = port.convertFtoC(temp);
            result2 = port.convertCtoF(temp);
        }
        out.println("Temperature conversion from F to C: " + result1);
    %>
    <br/>
    <%
        out.println("Temperature conversion from C to F: " + result2);
    %>
</body>
```

Save and run index file

Practical No. 3

Aim:

<u>Steps:</u>

Step 1: Install Flask: Make sure you have Python installed on your system.

Open cmd and type the following command

```
C:\Users\Kishore>pip install Flask
Requirement already satisfied: Flask in c:\users\kishore\appdata\local\programs\python\python312\lib\site-packages (3.0.3)
Requirement already satisfied: Werkzeug>=3.0.0 in c:\users\kishore\appdata\local\programs\python\python312\lib\site-packages (from Flask) (3.0.3)
Requirement already satisfied: Jinja2>=3.1.2 in c:\users\kishore\appdata\local\programs\python\python312\lib\site-packages (from Flask) (3.1.4)
Requirement already satisfied: itsdangerous>=2.1.2 in c:\users\kishore\appdata\local\programs\python\python312\lib\site-packages (from Flask) (2.2.0)
Requirement already satisfied: click>=8.1.3 in c:\users\kishore\appdata\local\programs\python\python312\lib\site-packages (from Flask) (8.1.7)
Requirement already satisfied: blinker>=1.6.2 in c:\users\kishore\appdata\local\programs\python\python312\lib\site-packages (from Flask) (1.8.2)
Requirement already satisfied: colorama in c:\users\kishore\appdata\local\programs\python\python312\lib\site-packages (from click>=8.1.3->Flask) (0.4.6)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\kishore\appdata\local\programs\python\python312\lib\site-packages (from Jinja2>=3.1.2->Flask) (2.1.5)

[notice] A new release of pip is available: 24.2 -> 24.3.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Step 2: Create a folder name SimpleRESTService, Open with VS Code and create python file.

Check and Install Extensions for Python

Code:

```
from flask import Flask, jsonify, request
app = Flask(__name__)

# Sample data
data = [
    {'id': 1, 'name': 'Item 1'},
    {'id': 2, 'name': 'Item 2'},
]

# Endpoint to get all items
@app.route('/items', methods=['GET'])
def get_items():
    return jsonify({'items': data})

# Endpoint to get a specific item by ID
@app.route('/items/<int:item_id>', methods=['GET'])
def get_item(item_id):
```

```
    item = next((item for item in data if item['id'] == item_id), None)
    if item:
        return jsonify({'item': item})
    else:
        return jsonify({'message': 'Item not found'}), 404

# Endpoint to add a new item
@app.route('/items', methods=['POST'])
def add_item():
    new_item = {'id': len(data) + 1, 'name': request.json['name']}
    data.append(new_item)
    return jsonify({'message': 'Item added successfully', 'item': new_item}), 201

# Run the application
if __name__ == '__main__':
    app.run(debug=True)
```

Output:

View>Terminal

Type

Path> **python app.py**

Demonstration:

Postman

API Testing Software Application to Test the API

Open Postman

 # Endpoint to get all items

Response:


```
{
   "items": [
      {
         "id": 1,
         "name": "Item 1"
      },
      {
         "id": 2,
         "name": "Item 2"
      }
   ]
}
```


Demonstration:

Throws Error when not item not found from Sample Data. Example

URL: http://127.0.0.1:5000/items/100

Demonstration:

 # Endpoint to add a new item

Send a POST Request:

URL: http://127.0.0.1:5000/items

Body: Set the body to raw JSON and include:

{"name":"item3"}

Response:

{

"item": {

"id": 3,

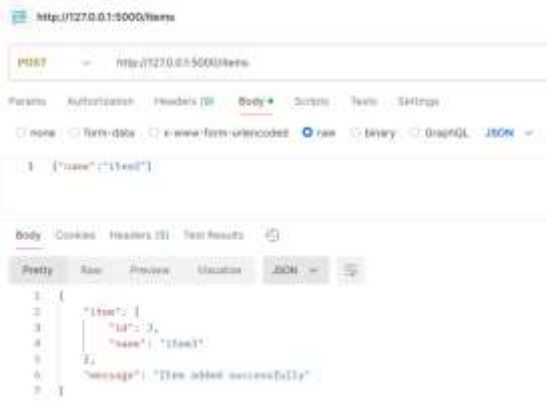
"name": "item3"

},

"message": "Item added successfully"

}

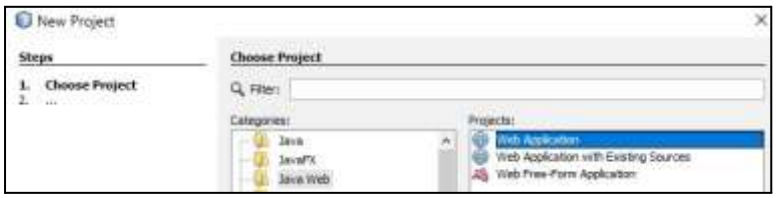
Demonstration:

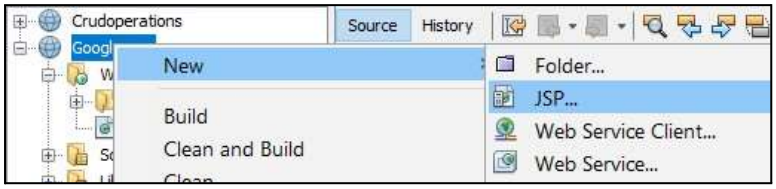


Practical No. 4

Aim: Develop application to consume Google's search / Google's Map RESTful Web service.

Step 1: First of all we need to create an Java Web Application with any name, let it be GoogleMap here.





Create input.jsp

```
        <%--
   Document   : input
   Created on : 15 Dec, 2025, 8:34:12 AM
   Author     : spdc
--%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
   <head>
      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
      <title>JSP Page</title>
   </head>
   <body>
      <form action="output.jsp">
         <pre>
Enter latitude: <input type="text" name="t1" />
Enter longitude: <input type="text" name="t2" />
<input type="submit" value="Show" />
         </pre>
      </form
</body>
   - </html>
```

We need the Google API key. The steps are shown here: - Visit Google APIs Console (https://console.developers.google.com), you have to login with your Google account).

Step 2: Click on the link → Press **Ctrl + o** → New Project → Project name → Create.
Agree & Continue.

Step 3: Enabling API → Click on "+ Enable APIs and services" → Maps Javascript APIs → **Enable.**

Step 4: Key Generation → Go for Credentials → Click on "+ Create Credentials" → API Keys → **Create** → Copy the created KEY.

Create output.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <style>        #map
{          height: 400px;
width: 100%;
      }
    </style>
  </head>
  <body>     <%        double lati =
Double.parseDouble(request.getParameter("t1"));        double longi =
Double.parseDouble(request.getParameter("t2"));     %>
    <h3> Google Maps </h3>
    <div id="map"></div>     <script lang="javascript">
        var marker = new google.maps.Marker({
position: info, map: map
        });
      }
    </script>
    <script defer
```

Step 5: Run the file → Right click on input.jsp → Run file

Enter latitude: 19.144333
Enter longitude: 72.939826
Show

Hit = "Show"

Output:

Practical No. 5

Aim: Installation and configuration of virtualization using KVM

<u>Steps:</u>

Step 1: sudo apt-get update

Step 2: egrep -c '(vmx|svm)' /proc/cpuinfo this command is use to check if Virtualization is enable or not if it enable it will give output 6 but if not then the value is 0

```
anurag@anurag-virtual-machine:~$ egrep -c '(vmx|svm)' /proc/cpuinfo
6
anurag@anurag-virtual-machine:~$
```

Step 3 :kvm-ok

```
anurag@anurag-virtual-machine:~$ kvm-ok
INFO: /dev/kvm exists
KVM acceleration can be used
anurag@anurag-virtual-machine:~$
```

Step 4: sudo apt install -y cpu-checker

```
root@anurag-virtual-machine:/home/anurag# sudo apt install -y cpu-checker
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
cpu-checker is already the newest version (0.7-1.3build1).
0 upgraded, 0 newly installed, 0 to remove and 276 not upgraded.
root@anurag-virtual-machine:/home/anurag#
```

Step 5: sudo apt-get install -y qemu-kvm virt-manager libvirt-daemon-system virtinst libvirt-clients bridge-utils

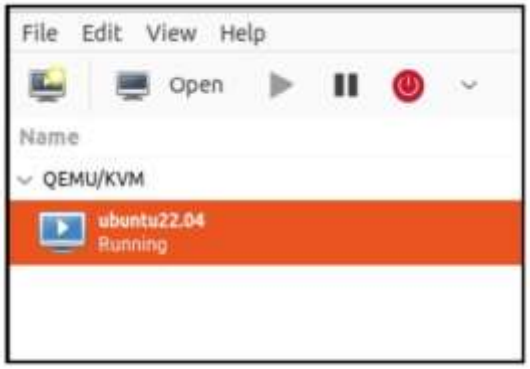
```
root@anurag-virtual-machine:/home/anurag# sudo apt-get install -y qemu-kvm virt-manager libvirt-daemon-system virtinst libvirt-clients bridge-utils
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Note, selecting 'qemu-system-x86' instead of 'qemu-kvm'
bridge-utils is already the newest version (1.7-1ubuntu3).
virt-manager is already the newest version (1:4.0.0-1).
virtinst is already the newest version (1:4.0.0-1).
libvirt-clients is already the newest version (8.0.0-1ubuntu7.8).
libvirt-daemon-system is already the newest version (8.0.0-1ubuntu7.8).
qemu-system-x86 is already the newest version (1:6.2+dfsg-2ubuntu6.17).
0 upgraded, 0 newly installed, 0 to remove and 276 not upgraded.
root@anurag-virtual-machine:/home/anurag#
```

Step 6: sudo systemctl enable –now libvirtd

sudo systemctl start libvirtd

sudo systemctl status linvirtd

Step 7: In this step search virtual machine and start it



Practical No. 6

Aim: Develop application to download image/video from server or upload image/video to server using MTOM techniques.

Steps:

Step 1: MTOM Client

Create an Index.jsp:

Index.jsp

```
<%@page import="java.io.BufferedOutputStream"%>
<%@page import="java.io.FileOutputStream"%>
<%@page import="java.io.FileInputStream"%>
<%@page import="java.io.BufferedInputStream"%>
<%@page import="javax.imageio.stream.FileImageInputStream"%>
<%@page import="java.io.File"%>
<%@page import="javax.xml.ws.soap.MTOMFeature"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<body>
<%-- start web service invocation --%><hr/>
<%
try {
pkg.ImageWS_Service service = new pkg.ImageWS_Service();
pkg.ImageWS port = service.getImageWSPort(new MTOMFeature(6000));
// TODO initialize WS operation arguments here
String filePath="D:/Image/image.jpg";
```

```
File file=new File(filePath);
FileInputStream fis=new FileInputStream(file);
BufferedInputStream bis=new BufferedInputStream(fis);
java.lang.String filename = file.getName();
byte[]imageBytes=new byte[(int)file.length()];
bis.read(imageBytes);
port.upload(filename, imageBytes);
bis.close();
out.println("File uploaded :"+filePath);
} catch (Exception ex) {
// TODO handle custom exceptions here
}
%>
<%-- end web service invocation --%><hr/>
<%-- start web service invocation --%><hr/>
<%
try {
pkg.ImageWS_Service service = new pkg.ImageWS_Service();
pkg.ImageWS port = service.getImageWSPort();
// TODO initialize WS operation arguments here
java.lang.String filename = "image.jpg";
String filePath="D:/Image/DOWNLOAD/"+filename;
// TODO process result here
byte[] fileBytes = port.download(filename);
FileOutputStream fos=new FileOutputStream(filePath);
BufferedOutputStream bos=new BufferedOutputStream(fos);
bos.write(fileBytes);
bos.close();
out.println("File downloaded :"+filePath);
```

```
} catch (Exception ex) {
// TODO handle custom exceptions here
}
%>
<%-- end web service invocation --%><hr/>
</body>
</html>
```

Step 2: MTOMServer

Create ImageWS.java

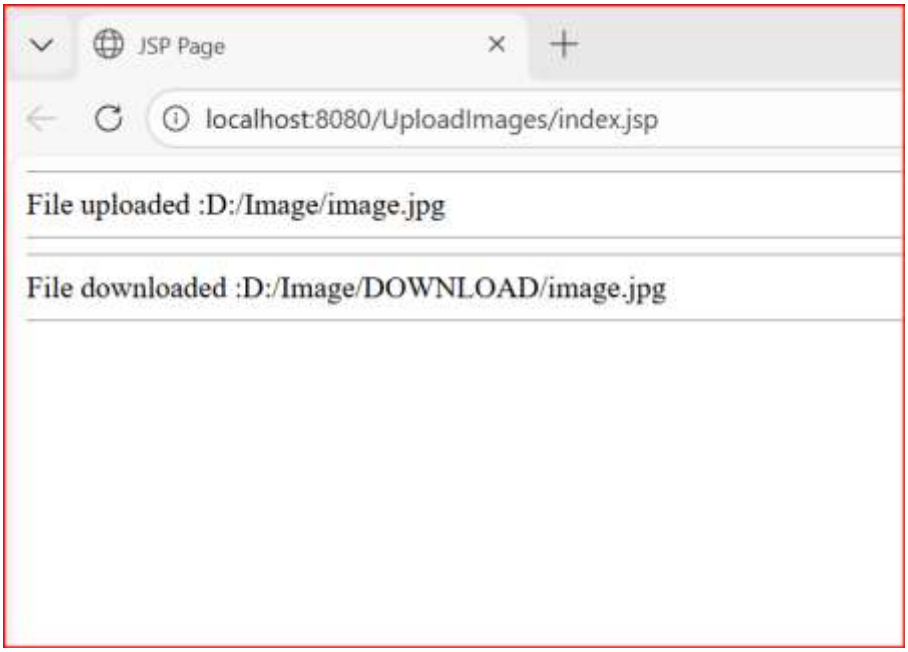
```
package mypkg;
import java.io.*;
import javax.jws.Oneway;
import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;
import javax.xml.ws.soap.MTOM;
@MTOM(enabled = true,threshold = 6000)
@WebService(serviceName = "ImageWS")

public class ImageWS {
   @WebMethod(operationName ="upload")
   @Oneway
   public void upload(@WebParam(name ="Filename")String Filename,
@WebParam(name="ImageBytes")byte[]ImageBytes)
   {
         String filePath="D:/Image/UPLOAD/"+Filename;
         try
```

```
    @WebMethod(operationName = "download")
    public byte[] download(@WebParam(name = "Filename") String Filename) {
        String filePath="D:/Image/UPLOAD/"+Filename;
        System.out.println("Sending file :"+filePath);
        catch(Exception ex)
        {
        System.out.println(ex);
        }
    return null;
    }
}
```

OUTPUT:-

Check in respective folder for uploaded and downloaded images



Practical No. 7

Aim: Implement FOSS-Cloud Functionality VSI (Virtual Server Infrastructure), Infrastructure as a Service (IaaS), Creating Virtual Machine or Storage.

FOSS Cloud Installation:

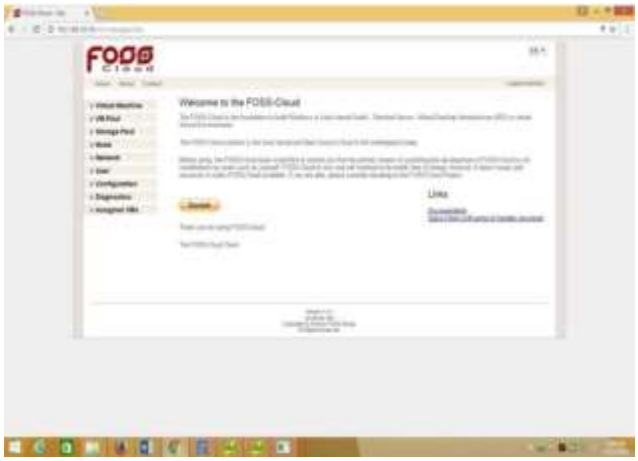
Steps:

1. Choose your keymap.
2. Confirm that you want to start: yes
3. Choose Demo-System: 1
4. Choose a Block-Device: sda
5. Confirm that you want to continue: yes
6. Confirm that you want to continue: yes
7. Choose the network interface: eth0
8. Choose if you want to use automatic network configuration: Yes
9. Reboot your system: yes
10. Login as root and run "fc-node-configuration -ndemo-system --password admin"

Login as admin

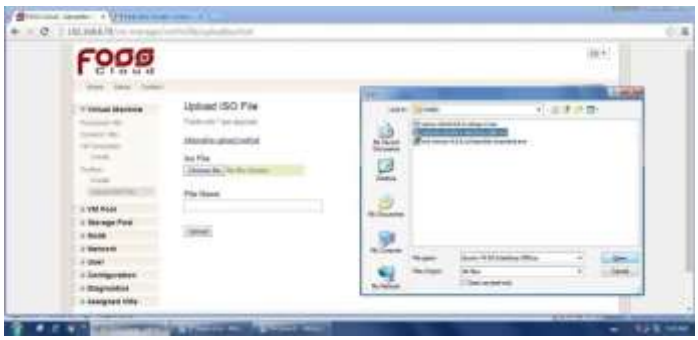
Password: admin

Home page of FOSS Cloud at Client



<u>Uploading ISOfiles to foss cloud</u>

Go to virtual machineàProfileàUpload ISO fileàfile name.



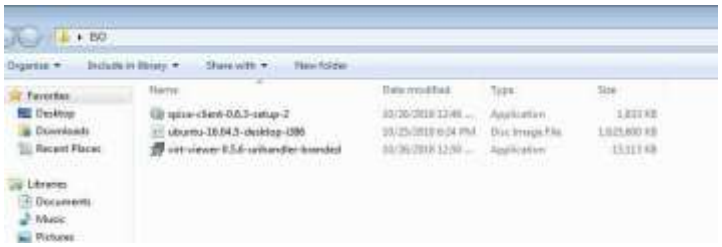
Creating a Profile:

- The profile creates the relationship between the ISO file and the FOSS-Cloud.
- Open Virtual Machines
- Choose vm-Profiles
- Choose the right Base Profile
- Choose the right architecture (Windows only x86_64)
- Chose the language (it is a information - not keyboard relevant)
- Choose the ISO file
- Fill out name and description
- Choose the amount of memory and volume capacity
- Choose amount of CPU
- Choose clock offset (normally Windows is "localtime" and Linux is "utc")

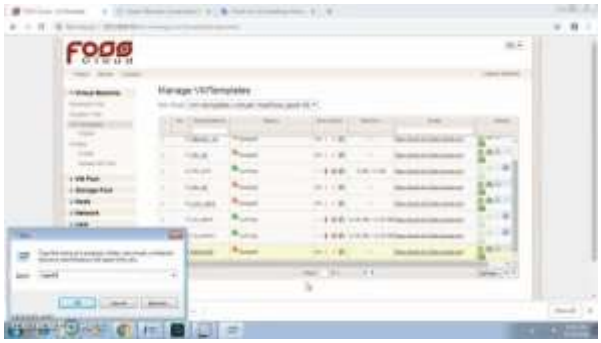
Creating Template:

- Choose the profile you have prepared before
- Add the vm-pool and one or more nodes, where you will run this vm (when the chosen vm-pool has only one nodeassigned, you don't have a choice)
- You can change all the other informations you have entered before

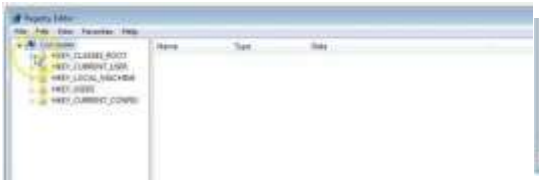
- **Install virt Viewer and spice**



- **Go to regedit**



- **Go to Computer à HKEY_CLASSES_ROOT à SPICE à SHELL à COMMAND**



- **Go to Default and click Modify**
- **Paste path ofvirt vieweràvirt_viewer_path %1(Actual path ofVirt Viewer)**
- **Run Your VM and View using VM Template.**

- **Open Browser in Ubuntu.**

Practical No. 8

Aim: Implement FOSS-Cloud Functionality - VSI Infrastructure as a Service (IaaS), Storage.

Steps:

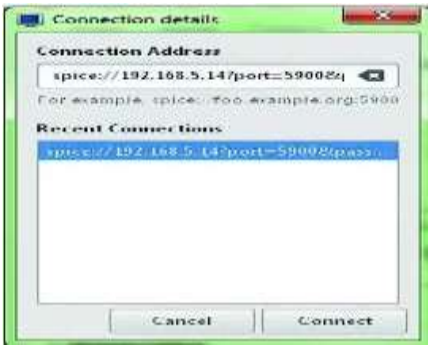
1. after installation, it will show you an ip address. put it im your browser to access your administrator page the default user credentials are user: admin and password: adimin. for root login - username- root and password - password.

2. The first screen after login shows many options to install and deploy any virtual machine. to install a virtual machine click on virtuai machine-> upload iso file option and upload the bootable iso file. here, we are going to upload linux elementary 0s is 0.

3. Once you uploaded the file, create vimtemplate. in this option you are basically configuring your virtual machine's storage location, cpu, memory, node etc. here, you will find single nodes ana vw po01 1 respective options because everything was installed at the single server.

4. Now, click on vmtemplates and you will see a template which you have created in step 3. to start your machine go to run action tab and click on the green arrow.
under status tab, it shows the running text with the green circle which shows that your machine is running without any errors. to view your virtual machine click on a blue square box under action tab.

5. To view your virtual machine you have to download spice client tool. the download link can be found under the links option. after downloading the application, click on the blue square to view machine. when you click on it, the browser will pop-up for launching the application.
   If the application does not launch automatically, launch it manually by entering the link and password in the remote viewer tool which you will see in the pop-up message.



6. Once it connects, enter the password which was given in the link and click ok.
7. Finally you will able to view and control your virtual machine.

Practical No. 10

Aim: Implementation of Openstack with user and private network creation.

1. In order to install devstack in a system first you have to create a linux VM on your computer or remotely on the cloud.
2. The VM must have at least 4GB of memory and proper internet connection is also important. Here we are using Ubuntu 24.
3. Follow the following steps to install the openstack in your VM.

Steps:

Step 1: **Update ubuntu System**

1. $sudo apt update
2. $sudo apt -upgrade
3. $sudo reboot

Step 2: **Add Stack User**

1. DevStack should be run as a non-root user with sudo enabled (standard logins to cloud images such as “ubuntu” or “cloud-user” are usually fine).
2. If you are not using a cloud image, you can create a separate *stack* user to run DevStack with sudo useradd -s /bin/bash -d /opt/stack -m stack

Step 3: **Since this user will be making many changes to your system, it should have sudo privileges**:

1. echo "stack ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/stack
2. You can switch to the stack user by running following command
   sudo su - stack

Step 5: **Create a devstack configuration file.**

1. cd devstack

2. Create a local.conf

   Create a **local.conf** file with four passwords and the host IP address

3. **[[local|localrc]]**

   ADMIN_PASSWORD=secret

   DATABASE_PASSWORD=$ADMIN_PASSWORD

   RABBIT_PASSWORD=$ADMIN_PASSWORD

   SERVICE_PASSWORD=$ADMIN_PASSWORD

   HOST_IP="Your machine IP address"

   This is the minimum required config to get started with DevStack.

Step 6: **Install Openstack with devstack**

1. execute following command, /stack.sh
2. Devstack will install the following components
   - Compute service(Nova)
   - Image service (Glance)
   - Identity Services (Keystone)
   - Block Storage Service (Cinder)
   - Object Storage(Swift)
   - Openstack Dashboard(Horizon)
   - Network Service (Neutron)
   - Placement API (Placement)

   This will take 15 - 30 minutes, largely depending on the speed of your internet connection. Many git trees and packages will be installed during this process. At very end of the installation you will get a host ipadddress,URL for managing it and the username and password to handle the administrative task

Step 7: **Accessing openstack on a browser**

1. Copy the horizon URL given in the installation output and paste it into your browser http://<ipaddress>/dashboard
2. Login with username-admin password -<given in local,conf file
3. Here you can create projects,users and assign them rolls.
4. Also you can create Virtua lMachine ,configure a network so that machines can communicate with each other and with an external environment.
5. In openstack creating VM's menace creating instances